

TRD

Technical Requirements Document (TRD)

AWS Glue Data Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Technical Requirements Document (TRD) translates the Functional Requirements Document (FRD) into implementation-ready technical specifications for an AWS Glue-based data pipeline. This document provides detailed technical design, data schemas, transformation logic, and runtime parameters necessary to generate production-ready code for customer order processing with SCD Type 2 implementation using Apache Hudi.

Scope

- In Scope:
 - Technical specifications for AWS Glue ETL jobs (Python/PySpark)
 - Data ingestion from S3 CSV sources to Spark DataFrames
 - Data quality transformations (null removal, deduplication)
 - AWS Glue Data Catalog table registration using Parquet format
 - Apache Hudi integration for SCD Type 2 implementation
 - Incremental processing logic using Glue job bookmarks
 - Aggregation logic for customer spend analytics
 - Error handling, logging, and validation specifications
 - Runtime parameter definitions
 - Schema mappings and transformations
- Out of Scope:
 - IAM role and policy definitions (assumed pre-configured)
 - S3 bucket creation and lifecycle policies
 - Network and VPC configurations
 - Glue Data Catalog database creation
 - CloudWatch alarm configurations
 - Athena query patterns and optimization
 - Data encryption key management
 - Lake Formation policy implementation

Assumptions

1. Data Types: Since the FRD does not specify data types for source columns, the following assumptions are made:
 - CustId: String
 - Name: String
 - EmailId: String
 - Region: String
 - OrderId: String
 - ItemName: String
 - PricePerUnit: Decimal(10,2)
 - Qty: Integer
 - Date: String (format: YYYY-MM-DD or similar parseable format)
2. File Naming: CSV files follow consistent naming patterns (e.g., customer_.csv, order_.csv)
3. CSV Format: CSV files contain header rows and use comma as delimiter
4. Null Representation: Only "Null" (string literal, case-sensitive) and NULL (actual null) require removal
5. Duplicate Detection: Duplicates are identified by exact match across all columns
6. Hudi Configuration:
 - Record key for ordersummary: CustId + OrderId
 - Precombine field: OpTs
 - Table type: COPY_ON_WRITE
 - Operation: UPSERT
7. Partitioning: No explicit partitioning strategy specified; will use non-partitioned tables unless performance requires optimization
8. Date Handling: Current timestamp for SCD Type 2 operations will use system time in UTC
9. Glue Version: AWS Glue 4.0 with Spark 3.3 and Python 3.10
10. Hudi Version: Hudi 0.12.x compatible with Glue 4.0
11. Job Bookmark: Enabled for incremental customer change detection
12. Write Mode: Catalog tables use dynamic overwrite; Hudi uses upsert
13. Logging Level: INFO level logging to CloudWatch

14. Error Threshold: Jobs fail on critical errors; warnings logged for data quality issues

•

2. Technical Requirements

TR-INGEST-001: Technical Implementation for Customer Data Ingestion

- Related Functional Requirement ID(s): FR-INGEST-001

- Objective:

Implement S3 CSV reader to load customer data into Spark DataFrame with schema validation and error handling.

- Source Details:

- Source Type: S3 CSV Files

- Source Path: s3://adif-sdlc/sdlc_wizard/customerdata/

- Source Format: CSV with header

- Source Pattern: All CSV files in the directory (.csv)

- Target Details:

- Target Type: In-memory Spark DataFrame

- Target Name: df_customer_raw

- Input Schema:

	Column Name	Data Type	Nullable	Description	
	-----	-----	-----	-----	
	CustId	String	No	Customer identifier	
	Name	String	No	Customer name	
	EmailId	String	No	Customer email address	
	Region	String	No	Customer region	

- Output Schema:

Same as Input Schema (in-memory DataFrame)

- Processing / Transformation Logic:

1. Initialize Spark session with Glue context

2. Use Glue DynamicFrame.from_options() or spark.read.csv() to read from S3

3. Specify CSV options:

- header=true

- inferSchema=false (use explicit schema)

- delimiter=","

4. Apply explicit schema definition matching Input Schema

5. Validate that all expected columns are present

6. Load into DataFrame named df_customer_raw

7. Log record count after load

- Validation Rules:

- File must exist at source path

- CSV must contain header row with exact column names: CustId, Name, EmailId, Region

- At least one data row must be present (empty files trigger warning)

- No schema drift allowed (column names and order must match)

- Error Handling and Logging:

- Error: S3 path does not exist → Log error "Source path not found:

- s3://adif-sdlc/sdlc_wizard/customerdata/" → Fail job with exit code 1

- Error: No CSV files found → Log error "No CSV files found at source path" → Fail job with exit code 1

- Error: Schema mismatch → Log error "Schema mismatch. Expected columns: [CustId, Name, EmailId, Region], Found: [actual columns]" → Fail job with exit code 1

- Warning: Empty CSV file → Log warning "Empty CSV file detected: [filename]" → Continue processing

- Info: Log "Successfully loaded [count] customer records from S3"

- Runtime Parameters:

- `SOURCE\_CUSTOMER\_PATH`: s3://adif-sdlc/sdlc_wizard/customerdata/

- `CSV\_DELIMITER`: ","

- `CSV\_HEADER`: true

- Operational Notes:

- Use Glue job bookmark to track processed files if incremental file processing is needed

- Consider using Glue DynamicFrame for better integration with Glue Data Catalog

- Ensure Glue job has s3:GetObject permission on source bucket

-

TR-INGEST-002: Technical Implementation for Order Data Ingestion

- Related Functional Requirement ID(s): FR-INGEST-002

- Objective:

Implement S3 CSV reader to load order data into Spark DataFrame with schema validation and error handling.

- Source Details:
- Source Type: S3 CSV Files
- Source Path: s3://adif-sdlc/sdlc_wizard/orderdata/
- Source Format: CSV with header
- Source Pattern: All CSV files in the directory (.csv)

- Target Details:
- Target Type: In-memory Spark DataFrame
- Target Name: df_order_raw

- Input Schema:

	Column Name	Data Type	Nullable	Description	
	-----	-----	-----	-----	
	OrderId	String	No	Order identifier	
	ItemName	String	No	Product/item name	
	PricePerUnit	Decimal(10,2)	No	Price per unit	
	Qty	Integer	No	Quantity ordered	
	Date	String	No	Order date	
	CustId	String	No	Customer identifier (FK)	

- Output Schema:

Same as Input Schema (in-memory DataFrame)

- Processing / Transformation Logic:
 1. Initialize Spark session with Glue context
 2. Use Glue DynamicFrame.from_options() or spark.read.csv() to read from S3
 3. Specify CSV options:
 - header=true
 - inferSchema=false (use explicit schema)
 - delimiter=","
 4. Apply explicit schema definition matching Input Schema
 5. Validate that all expected columns are present
 6. Load into DataFrame named df_order_raw
 7. Log record count after load
- Validation Rules:
 - File must exist at source path

- CSV must contain header row with exact column names: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- At least one data row must be present (empty files trigger warning)
- No schema drift allowed (column names and order must match)
- PricePerUnit must be parseable as Decimal(10,2)
- Qty must be parseable as Integer
- Error Handling and Logging:
 - Error: S3 path does not exist → Log error "Source path not found: s3://adif-sdlc/sdlc_wizard/orderdata/" → Fail job with exit code 1
 - Error: No CSV files found → Log error "No CSV files found at source path" → Fail job with exit code 1
 - Error: Schema mismatch → Log error "Schema mismatch. Expected columns: [OrderId, ItemName, PricePerUnit, Qty, Date, CustId], Found: [actual columns]" → Fail job with exit code 1
 - Error: Type casting failure → Log error "Type casting failed for column [column_name] at row [row_number]" → Fail job with exit code 1
 - Warning: Empty CSV file → Log warning "Empty CSV file detected: [filename]" → Continue processing
 - Info: Log "Successfully loaded [count] order records from S3"
- Runtime Parameters:
 - `SOURCE\_ORDER\_PATH`: s3://adif-sdlc/sdlc_wizard/orderdata/
 - `CSV\_DELIMITER`: ","
 - `CSV\_HEADER`: true
- Operational Notes:
 - Use Glue job bookmark to track processed files if incremental file processing is needed
 - Consider using Glue DynamicFrame for better integration with Glue Data Catalog
 - Ensure Glue job has s3:GetObject permission on source bucket
-

TR-CLEAN-001: Technical Implementation for Null Value Removal

- Related Functional Requirement ID(s): FR-CLEAN-001
- Objective:

Implement data quality filter to remove records containing string literal "Null" or actual NULL values from customer and order DataFrames.

- Source Details:
- Source Type: In-memory Spark DataFrame

- Source Name: df_customer_raw, df_order_raw
- Target Details:
- Target Type: In-memory Spark DataFrame
- Target Name: df_customer_clean_nulls, df_order_clean_nulls
- Input Schema:
- Customer: Same as TR-INGEST-001 Output Schema
- Order: Same as TR-INGEST-002 Output Schema
- Output Schema:

Same as Input Schema (filtered DataFrames)

- Processing / Transformation Logic:

1. For df_customer_raw:

- Create filter condition:

...

```
(col("CustId") != "Null") & (col("CustId").isNotNull()) &
(col("Name") != "Null") & (col("Name").isNotNull()) &
(col("EmailId") != "Null") & (col("EmailId").isNotNull()) &
(col("Region") != "Null") & (col("Region").isNotNull())
```

...

- Apply filter to create df_customer_clean_nulls
- Calculate removed_count = df_customer_raw.count() - df_customer_clean_nulls.count()
- Log removed_count

2. For df_order_raw:

- Create filter condition:

...

```
(col("OrderId") != "Null") & (col("OrderId").isNotNull()) &
(col("ItemName") != "Null") & (col("ItemName").isNotNull()) &
(col("PricePerUnit") != "Null") & (col("PricePerUnit").isNotNull()) &
(col("Qty") != "Null") & (col("Qty").isNotNull()) &
(col("Date") != "Null") & (col("Date").isNotNull()) &
(col("CustId") != "Null") & (col("CustId").isNotNull())
```

...

- Apply filter to create df_order_clean_nulls
- Calculate removed_count = df_order_raw.count() - df_order_clean_nulls.count()

- Log removed_count
- Validation Rules:
 - String comparison is case-sensitive ("Null" only, not "null" or "NULL")
 - All columns must be checked for both string "Null" and actual NULL
 - If all records are removed, log warning but continue (empty DataFrame is valid)
- Error Handling and Logging:
 - Warning: All customer records removed → Log warning "All customer records contained null values. Resulting DataFrame is empty." → Continue processing
 - Warning: All order records removed → Log warning "All order records contained null values. Resulting DataFrame is empty." → Continue processing
 - Info: Log "Removed [count] customer records containing null values. Remaining: [count]"
 - Info: Log "Removed [count] order records containing null values. Remaining: [count]"
 - Info: If no nulls found → Log "No null values found in customer/order data"
- Runtime Parameters:
 - `NULL\_STRING\_LITERAL`: "Null"
- Operational Notes:
 - Use PySpark Column API for filter conditions
 - Consider using dropna() for actual NULL removal combined with string literal filter
 - Performance: Filter operations are lazy; actual execution occurs on action (count, write)
-

TR-CLEAN-002: Technical Implementation for Duplicate Removal

- Related Functional Requirement ID(s): FR-CLEAN-002
- Objective:

Implement deduplication logic to remove duplicate records based on all columns from customer and order DataFrames.

- Source Details:
 - Source Type: In-memory Spark DataFrame
 - Source Name: df_customer_clean_nulls, df_order_clean_nulls
- Target Details:
 - Target Type: In-memory Spark DataFrame
 - Target Name: df_customer_clean, df_order_clean

- Input Schema:
- Customer: Same as TR-CLEAN-001 Output Schema
- Order: Same as TR-CLEAN-001 Output Schema
- Output Schema:

Same as Input Schema (deduplicated DataFrames)

- Processing / Transformation Logic:
 1. For df_customer_clean_nulls:
 - Apply dropDuplicates() on all columns: ["CustId", "Name", "EmailId", "Region"]
 - Store result in df_customer_clean
 - Calculate removed_count = df_customer_clean_nulls.count() - df_customer_clean.count()
 - Log removed_count
 2. For df_order_clean_nulls:
 - Apply dropDuplicates() on all columns: ["OrderId", "ItemName", "PricePerUnit", "Qty", "Date", "CustId"]
 - Store result in df_order_clean
 - Calculate removed_count = df_order_clean_nulls.count() - df_order_clean.count()
 - Log removed_count
 3. Validation after deduplication:
 - Check if df_customer_clean.count() == 0 → Fail job
 - Check if df_order_clean.count() == 0 → Fail job
- Validation Rules:
 - Duplicate detection is based on exact match across all columns
 - At least one record must remain after deduplication
 - First occurrence of duplicate is retained (Spark default behavior)
- Error Handling and Logging:
 - Error: All customer records are duplicates (count = 0) → Log error "All customer records were duplicates. No unique records remain." → Fail job with exit code 1
 - Error: All order records are duplicates (count = 0) → Log error "All order records were duplicates. No unique records remain." → Fail job with exit code 1
 - Info: Log "Removed [count] duplicate customer records. Remaining: [count]"
 - Info: Log "Removed [count] duplicate order records. Remaining: [count]"
 - Info: If no duplicates found → Log "No duplicate records found in customer/order data"
- Runtime Parameters:

None (uses default Spark behavior)

- Operational Notes:
- `dropDuplicates()` is a transformation operation (lazy evaluation)
- For large datasets, consider repartitioning before deduplication
- Spark retains first occurrence by default; order is not guaranteed unless explicitly sorted
-

TR-CATALOG-001: Technical Implementation for Customer Table Catalog Registration

- Related Functional Requirement ID(s): FR-CATALOG-001
- Objective:

Write cleaned customer data to S3 in Parquet format and register as a queryable table in AWS Glue Data Catalog.

- Source Details:
- Source Type: In-memory Spark DataFrame
- Source Name: `df_customer_clean`
- Target Details:
- Target Type: S3 Parquet Files + Glue Catalog Table
- Target S3 Path: `s3://adif-sdlc/curated/sdlc_wizard/customer/`
- Target Database: `gen_ai_poc_databricksco`
- Target Table: `sdlc_wizard_customer`
- Target Format: Parquet
- Input Schema:

	Column Name	Data Type	Nullable	Description	
	-----	-----	-----	-----	
	CustId	String	No	Customer identifier	
	Name	String	No	Customer name	
	EmailId	String	No	Customer email address	
	Region	String	No	Customer region	

- Output Schema:

Same as Input Schema (persisted in Parquet format)

- Processing / Transformation Logic:
- 1. Write `df_customer_clean` to S3:
- Use `df.write.format("parquet")`

- Mode: overwrite
- Path: s3://adif-sdlc/curated/sdlc_wizard/customer/
- Compression: snappy (default)

2. Register table in Glue Data Catalog:

- Use Glue DynamicFrame.from_catalog() or Glue API
- Database: gen_ai_poc_databrickscoe
- Table: sdlc_wizard_customer
- Location: s3://adif-sdlc/curated/sdlc_wizard/customer/
- Format: parquet
- Update table metadata and statistics

3. Verify table registration:

- Query Glue Catalog to confirm table exists
- Log table location and record count
- Validation Rules:
 - Database gen_ai_poc_databrickscoe must exist before table registration
 - S3 write must complete successfully before catalog registration
 - Table schema in catalog must match DataFrame schema
 - Table must be queryable via Athena after registration
- Error Handling and Logging:
 - Error: Database does not exist → Log error "Glue Data Catalog database 'gen_ai_poc_databrickscoe' not found" → Fail job with exit code 1
 - Error: S3 write fails → Log error "Failed to write customer data to S3: [error_message]" → Fail job with exit code 1
 - Error: Catalog registration fails → Log error "Failed to register table in Glue Catalog: [error_message]" → Attempt rollback of S3 write → Fail job with exit code 1
 - Info: Log "Successfully wrote [count] customer records to s3://adif-sdlc/curated/sdlc_wizard/customer/"
 - Info: Log "Successfully registered table gen_ai_poc_databrickscoe.sdlc_wizard_customer"
- Runtime Parameters:
 - `TARGET\_CUSTOMER\_S3\_PATH`: s3://adif-sdlc/curated/sdlc_wizard/customer/
 - `GLUE\_DATABASE`: gen_ai_poc_databrickscoe
 - `CUSTOMER\_TABLE\_NAME`: sdlc_wizard_customer
 - `WRITE\_MODE`: overwrite
 - `FILE\_FORMAT`: parquet

- Operational Notes:
- Use Glue `DynamicFrame.write_dynamic_frame.from_catalog()` for seamless catalog integration
- Ensure Glue job has `s3:PutObject` permission on target bucket
- Ensure Glue job has `glue:CreateTable` and `glue:UpdateTable` permissions
- Parquet format provides columnar storage and compression benefits for Athena queries
- Consider partitioning by Region if query patterns favor region-based filtering (not specified in FRD)
-

TR-CATALOG-002: Technical Implementation for Order Table Catalog Registration

- Related Functional Requirement ID(s): FR-CATALOG-002
- Objective:

Write cleaned order data to S3 in Parquet format and register as a queryable table in AWS Glue Data Catalog.

- Source Details:
- Source Type: In-memory Spark DataFrame
- Source Name: `df_order_clean`
- Target Details:
- Target Type: S3 Parquet Files + Glue Catalog Table
- Target S3 Path: `s3://adif-sdlc/curated/sdlc_wizard/order/`
- Target Database: `gen_ai_poc_databrickscoe`
- Target Table: `sdlc_wizard_order`
- Target Format: Parquet
- Input Schema:

	Column Name	Data Type	Nullable	Description	
	-----	-----	-----	-----	
	OrderId	String	No	Order identifier	
	ItemName	String	No	Product/item name	
	PricePerUnit	Decimal(10,2)	No	Price per unit	
	Qty	Integer	No	Quantity ordered	
	Date	String	No	Order date	
	CustId	String	No	Customer identifier (FK)	

- Output Schema:

Same as Input Schema (persisted in Parquet format)

- Processing / Transformation Logic:

1. Write df_order_clean to S3:

- Use df.write.format("parquet")
- Mode: overwrite
- Path: s3://adif-sdlc/curated/sdlc_wizard/order/
- Compression: snappy (default)

2. Register table in Glue Data Catalog:

- Use Glue DynamicFrame.from_catalog() or Glue API
- Database: gen_ai_poc_databricksco
- Table: sdlc_wizard_order
- Location: s3://adif-sdlc/curated/sdlc_wizard/order/
- Format: parquet
- Update table metadata and statistics

3. Verify table registration:

- Query Glue Catalog to confirm table exists
- Log table location and record count

- Validation Rules:

- Database gen_ai_poc_databricksco must exist before table registration
- S3 write must complete successfully before catalog registration
- Table schema in catalog must match DataFrame schema
- Table must be queryable via Athena after registration

- Error Handling and Logging:

- Error: Database does not exist → Log error "Glue Data Catalog database 'gen_ai_poc_databricksco' not found" → Fail job with exit code 1
- Error: S3 write fails → Log error "Failed to write order data to S3: [error_message]" → Fail job with exit code 1
- Error: Catalog registration fails → Log error "Failed to register table in Glue Catalog: [error_message]" → Attempt rollback of S3 write → Fail job with exit code 1
- Info: Log "Successfully wrote [count] order records to s3://adif-sdlc/curated/sdlc_wizard/order/"
- Info: Log "Successfully registered table gen_ai_poc_databricksco.sdlc_wizard_order"

- Runtime Parameters:

- `TARGET\_ORDER\_S3\_PATH`: s3://adif-sdlc/curated/sdlc_wizard/order/

- `GLUE\_DATABASE`: gen_ai_poc_databricksco
- `ORDER\_TABLE\_NAME`: sdlc_wizard_order
- `WRITE\_MODE`: overwrite
- `FILE\_FORMAT`: parquet
- Operational Notes:
 - Use Glue DynamicFrame.write_dynamic_frame.from_catalog() for seamless catalog integration
 - Ensure Glue job has s3:PutObject permission on target bucket
 - Ensure Glue job has glue:CreateTable and glue:UpdateTable permissions
 - Parquet format provides columnar storage and compression benefits for Athena queries
 - Consider partitioning by Date if query patterns favor date-based filtering (not specified in FRD)
-

TR-SCD2-001: Technical Implementation for SCD Type 2 Order Summary

- Related Functional Requirement ID(s): FR-SCD2-001
- Objective:

Implement SCD Type 2 logic using Apache Hudi to maintain historical changes in order summary table with proper versioning and time-based tracking.

- Source Details:
 - Source Type: In-memory Spark DataFrame (joined data)
 - Source Name: df_customer_clean (from TR-CATALOG-001), df_order_clean (from TR-CATALOG-002)
 - Existing Hudi Table: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (if exists)
- Target Details:
 - Target Type: S3 Hudi Table + Glue Catalog Table
 - Target S3 Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
 - Target Database: gen_ai_poc_databricksco
 - Target Table: ordersummary
 - Target Format: Hudi (COPY_ON_WRITE)
- Input Schema (Joined Data):

	Column Name	Data Type	Nullable	Source	Description	
	-----	-----	-----	-----	-----	
	CustId	String	No	customer.CustId	Customer identifier	
	Name	String	No	customer.Name	Customer name	

	EmailId	String	No	customer.EmailId	Customer email	
	Region	String	No	customer.Region	Customer region	
	OrderId	String	No	order.OrderId	Order identifier	
	ItemName	String	No	order.ItemName	Product name	
	PricePerUnit	Decimal(10,2)	No	order.PricePerUnit	Unit price	
	Qty	Integer	No	order.Qty	Quantity	
	Date	String	No	order.Date	Order date	

- Output Schema (Hudi Table with SCD2 Columns):

	Column Name	Data Type	Nullable	Description	
	-----	-----	-----	-----	
	CustId	String	No	Customer identifier (part of record key)	
	OrderId	String	No	Order identifier (part of record key)	
	Name	String	No	Customer name	
	EmailId	String	No	Customer email	
	Region	String	No	Customer region	
	ItemName	String	No	Product name	
	PricePerUnit	Decimal(10,2)	No	Unit price	
	Qty	Integer	No	Quantity	
	Date	String	No	Order date	
	IsActive	Boolean	No	SCD2: Current record flag	
	StartDate	Timestamp	No	SCD2: Record effective start timestamp	
	EndDate	Timestamp	Yes	SCD2: Record effective end timestamp (null for active)	
	OpTs	Timestamp	No	SCD2: Operation timestamp (precombine field)	

- Processing / Transformation Logic:

1. Join customer and order data:

```
---
```

```
df_joined = df_customer_clean.join(df_order_clean, "CustId", "inner")
```

```
---
```

2. Add SCD2 columns to new records:

```
---
```

```
current_timestamp = current_timestamp()
```

```
df_new_records = df_joined.withColumn("IsActive", lit(True))
```

```
.withColumn("StartDate", current_timestamp)
```

```
.withColumn("EndDate", lit(None).cast(TimestampType()))
```

```
.withColumn("OpTs", current_timestamp)
```

...

3. Read existing Hudi table (if exists):

...

```
if hudi_table_exists(s3://adif-sdlc/curated/sdlc_wizard/ordersummary/):
    df_existing = spark.read.format("hudi").load(s3://adif-sdlc/curated/sdlc_wizard/ordersummary/)
else:
    df_existing = empty DataFrame with output schema
```

...

4. Identify changes (compare new vs existing active records):

- Filter existing active records: `df_existing_active = df_existing.filter(col("IsActive") == True)`
- Join new records with existing active on composite key (CustId + OrderId)
- Identify:
 - New records: Records in `df_new_records` not in `df_existing_active`
 - Changed records: Records in both with different values in any attribute (Name, EmailId, Region, ItemName, PricePerUnit, Qty, Date)
 - Unchanged records: Records in both with identical values

5. Prepare SCD2 updates:

- For new records: Insert as-is with `IsActive=True`, `StartDate=current_timestamp`, `EndDate=null`, `OpTs=current_timestamp`
- For changed records:
 - Create "close" records: Copy from `df_existing_active`, set `IsActive=False`, `EndDate=current_timestamp`, `OpTs=current_timestamp`
 - Create "insert" records: Copy from `df_new_records`, set `IsActive=True`, `StartDate=current_timestamp`, `EndDate=null`, `OpTs=current_timestamp`
- For unchanged records: No action (not included in upsert)

6. Combine records for Hudi upsert:

...

```
df_upsert = union(new_records, changed_close_records, changed_insert_records)
```

...

7. Write to Hudi:

...

```
hudi_options = {
    'hoodie.table.name': 'ordersummary',
    'hoodie.datasource.write.recordkey.field': 'CustId,OrderId',
```

```

'hoodie.datasource.write.precombine.field': 'OpTs',
'hoodie.datasource.write.partitionpath.field': '',
'hoodie.datasource.write.table.type': 'COPY_ON_WRITE',
'hoodie.datasource.write.operation': 'upsert',
'hoodie.datasource.write.hive_style_partitioning': 'false',
'hoodie.datasource.hive_sync.enable': 'true',
'hoodie.datasource.hive_sync.database': 'gen_ai_poc_databrickscoe',
'hoodie.datasource.hive_sync.table': 'ordersummary',
'hoodie.datasource.hive_sync.partition_extractor_class': 'org.apache.hudi.hive.NonPartitionedExtractor',
'hoodie.datasource.hive_sync.mode': 'hms'
}

df_upsert.write.format("hudi") \
.options(hudi_options) \
.mode("append") \
.save("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")
...

```

8. Verify SCD2 integrity:

- Query Hudi table to ensure no duplicate active records for same key
- Validate StartDate <= EndDate for all closed records
- Log counts: new records, updated records, total active records
- Validation Rules:
 - Composite key (CustId + OrderId) must be unique for active records (IsActive = True)
 - For all records where IsActive = False, EndDate must not be null
 - For all records where IsActive = True, EndDate must be null
 - StartDate must be <= EndDate for all closed records
 - OpTs must be populated for all records (used by Hudi for precombine)
 - Join between customer and order must produce at least one record (empty join triggers warning)
- Error Handling and Logging:
 - Error: Join produces no records → Log warning "Customer-Order join produced no records. Ordersummary table will not be updated." → Continue (skip Hudi write)
 - Error: Multiple active records for same key after upsert → Log error "Data integrity violation: Multiple active records found for key [CustId, OrderId]" → Fail job with exit code 1
 - Error: StartDate > EndDate detected → Log error "Data integrity violation: StartDate > EndDate for record [details]" → Fail job with exit code 1

- Error: Hudi write fails → Log error "Failed to write to Hudi table: [error_message]" → Fail job with exit code 1
- Info: Log "Identified [count] new records, [count] changed records, [count] unchanged records"
- Info: Log "Successfully upserted [count] records to ordersummary Hudi table"
- Info: Log "Total active records in ordersummary: [count]"
- Runtime Parameters:
 - `TARGET\_ORDERSUMMARY\_S3\_PATH`: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
 - `GLUE\_DATABASE`: gen_ai_poc_databrickscoe
 - `ORDERSUMMARY\_TABLE\_NAME`: ordersummary
 - `HUDI\_TABLE\_TYPE`: COPY_ON_WRITE
 - `HUDI\_RECORD\_KEY`: CustId,OrderId
 - `HUDI\_PRECOMBINE\_FIELD`: OpTs
 - `HUDI\_OPERATION`: upsert
- Operational Notes:
 - Hudi COPY_ON_WRITE provides snapshot isolation and is optimized for read-heavy workloads
 - Record key is composite (CustId + OrderId) to uniquely identify order-customer combinations
 - Precombine field (OpTs) determines which record to keep when multiple updates exist for same key
 - Hudi Hive sync automatically updates Glue Catalog metadata
 - Ensure Glue job has sufficient memory and DPUs for Hudi operations (recommend 10+ DPUs)
 - First run will create Hudi table; subsequent runs will perform upserts
 - Consider enabling Hudi compaction for long-running tables
-

TR-SCD2-002: Technical Implementation for Incremental Customer Update Processing

- Related Functional Requirement ID(s): FR-SCD2-002
- Objective:

Implement incremental processing logic to detect customer data changes and trigger SCD Type 2 updates only for affected order summary records.

- Source Details:
 - Source Type: S3 CSV Files + Glue Job Bookmark
 - Source Path: s3://adif-sdlc/sdlc_wizard/customerdata/
 - Bookmark Tracking: Glue job bookmark enabled to track processed customer files

- Target Details:
- Target Type: Trigger for TR-SCD2-001 processing
- Affected Records: Order summary records for changed customers only
- Input Schema:

	Column Name	Data Type	Nullable	Description	
	-----	-----	-----	-----	
	CustId	String	No	Customer identifier	
	Name	String	No	Customer name	
	EmailId	String	No	Customer email address	
	Region	String	No	Customer region	

- Output Schema:

Not applicable (triggers downstream processing)

- Processing / Transformation Logic:

1. Load current customer data:

- Read from s3://adif-sdlc/sdlc_wizard/customerdata/ (df_customer_current)
- Apply cleaning operations (TR-CLEAN-001, TR-CLEAN-002)

2. Load previous customer data (if exists):

- Check if Glue job bookmark exists
- If first run: Set df_customer_previous = empty DataFrame
- If subsequent run: Read previously cataloged customer table from Glue Catalog (gen_ai_poc_databrickscoe.sdlc_wizard_customer)

3. Identify changed customers:

...

if df_customer_previous is empty:

First run: all customers are "new"

df_changed_customers = df_customer_current

else:

Compare current vs previous

df_changed_customers = df_customer_current.subtract(df_customer_previous)

...

4. Filter orders for changed customers:

...

df_order_clean = load_and_clean_orders() # From TR-INGEST-002, TR-CLEAN-001, TR-CLEAN-002

```
df_affected_orders = df_order_clean.join(df_changed_customers.select("CustId"), "CustId", "inner")
...
```

5. Trigger SCD2 processing:

- If `df_changed_customers.count() == 0`:
- Log "No customer changes detected. Skipping SCD2 processing."
- Skip TR-SCD2-001 execution
- Else:
- Execute TR-SCD2-001 with `df_changed_customers` and `df_affected_orders`
- Log "Processing SCD2 updates for [count] changed customers affecting [count] orders"

6. Update job bookmark:

- Glue job bookmark automatically tracks processed S3 files
- Ensure bookmark is committed after successful processing
- Validation Rules:
- Customer change detection must handle first run scenario (no previous data)
- Changed customer identification is based on any attribute change (Name, EmailId, Region) for a given CustId
- If no changes detected, SCD2 processing should be skipped (optimization)
- Job bookmark must be enabled in Glue job configuration
- Error Handling and Logging:
- Info: First run detected → Log "First run: No previous customer data found. Processing all customers as new."
- Info: No changes detected → Log "No customer changes detected. Skipping SCD2 processing." → Exit successfully
- Info: Changes detected → Log "Detected [count] changed customers. Triggering SCD2 processing for [count] affected orders."
- Error: Unable to read previous customer data → Log warning "Unable to read previous customer data. Falling back to full refresh mode." → Process all customers
- Error: Job bookmark commit fails → Log error "Failed to commit job bookmark: [error_message]" → Fail job with exit code 1
- Runtime Parameters:
- ``ENABLE_INCREMENTAL_PROCESSING``: true
- ``GLUE_JOB_BOOKMARK``: job-bookmark-enable
- ``CUSTOMER_TABLE_NAME``: sdlc_wizard_customer
- ``GLUE_DATABASE``: gen_ai_poc_databrickscoe

- Operational Notes:
- Enable Glue job bookmark in job configuration: `--job-bookmark-option job-bookmark-enable`
- Job bookmark tracks processed S3 files and prevents reprocessing
- First run processes all customers; subsequent runs process only new/changed files
- If bookmark is reset or disabled, job will reprocess all data (full refresh)
- Consider using DataFrame caching for df_customer_previous if used multiple times
- Incremental processing significantly reduces processing time for large datasets
-

TR-AGG-001: Technical Implementation for Customer Aggregate Spend Table

- Related Functional Requirement ID(s): FR-AGG-001
- Objective:

Generate aggregated customer spend analytics table by calculating total spending per customer per date from active order summary records.

- Source Details:
- Source Type: Hudi Table (via Glue Catalog)
- Source Database: gen_ai_poc_databricksco
- Source Table: ordersummary
- Source Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Target Details:
- Target Type: S3 Parquet Files + Glue Catalog Table
- Target S3 Path: s3://adif-sdlc/analytics/customeraggregatespend/
- Target Database: gen_ai_poc_databricksco
- Target Table: customeraggregatespend
- Target Format: Parquet
- Input Schema (Active Order Summary Records):

	Column Name	Data Type	Nullable	Description	
	-----	-----	-----	-----	
	Name	String	No	Customer name	
	PricePerUnit	Decimal(10,2)	No	Unit price	
	Qty	Integer	No	Quantity	
	Date	String	No	Order date	
	IsActive	Boolean	No	SCD2 active flag	

- Output Schema:

	Column Name	Data Type	Nullable	Description	Derivation	
	-----	-----	-----	-----	-----	
	Name	String	No	Customer name	Direct map from ordersummary.Name	
	TotalAmount	Decimal(20,2)	No	Total spending	SUM(PricePerUnit × Qty)	
	Date	String	No	Order date	Direct map from ordersummary.Date	

- Processing / Transformation Logic:

1. Read active order summary records:

```
---
```

```
df_ordersummary = spark.read.format("hudi").load("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")
df_active_orders = df_ordersummary.filter(col("IsActive") == True)
```

```
---
```

2. Calculate TotalAmount per order:

```
---
```

```
df_with_total = df_active_orders.withColumn("TotalAmount", col("PricePerUnit") col("Qty"))
```

```
---
```

3. Aggregate by customer Name and Date:

```
---
```

```
df_aggregated = df_with_total.groupBy("Name", "Date") \
    .agg(sum("TotalAmount").alias("TotalAmount"))
```

```
---
```

4. Select final columns:

```
---
```

```
df_final = df_aggregated.select("Name", "TotalAmount", "Date")
```

```
---
```

5. Write to S3 in Parquet format:

```
---
```

```
df_final.write.format("parquet") \
    .mode("overwrite") \
    .save("s3://adif-sdlc/analytics/customeraggregatespend/")
```

```
---
```

6. Register table in Glue Data Catalog:

- Database: gen_ai_poc_databrickscoe
- Table: customeraggregatespend
- Location: s3://adif-sdlc/analytics/customeraggregatespend/
- Format: parquet

7. Log aggregation statistics:

- Total customers in aggregate: `df_final.select("Name").distinct().count()`
- Total date periods: `df_final.select("Date").distinct().count()`
- Total aggregate records: `df_final.count()`

• Validation Rules:

- Only active records (IsActive = True) from ordersummary should be included
- TotalAmount must not be negative (log warning if negative values detected)
- Aggregation must produce at least one record (empty result triggers warning)
- Output schema must match exactly: Name (String), TotalAmount (Decimal), Date (String)

• Error Handling and Logging:

- Warning: Ordersummary table is empty → Log warning "Ordersummary table is empty. Creating empty customeraggregatespend table." → Create empty table with schema
- Warning: No active records in ordersummary → Log warning "No active records found in ordersummary. Creating empty customeraggregatespend table." → Create empty table
- Warning: Negative TotalAmount detected → Log warning "Negative TotalAmount detected for customer [Name] on date [Date]: [amount]" → Include in output
- Info: Log "Aggregated spending for [count] unique customers across [count] date periods"
- Info: Log "Total aggregate records: [count]"
- Error: S3 write fails → Log error "Failed to write customeraggregatespend to S3: [error_message]" → Fail job with exit code 1
- Error: Catalog registration fails → Log error "Failed to register customeraggregatespend table: [error_message]" → Fail job with exit code 1

• Runtime Parameters:

- ``SOURCE_ORDERSUMMARY_PATH``: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- ``TARGET_AGGREGATE_S3_PATH``: s3://adif-sdlc/analytics/customeraggregatespend/
- ``GLUE_DATABASE``: gen_ai_poc_databrickscoe
- ``AGGREGATE_TABLE_NAME``: customeraggregatespend
- ``WRITE_MODE``: overwrite
- ``FILE_FORMAT``: parquet

• Operational Notes:

- Aggregation is performed only on active records to reflect current state
- TotalAmount calculation: $\text{PricePerUnit} \times \text{Qty}$ (per order line)
- Aggregation level: Customer Name + Date (one row per customer per date)
- Overwrite mode ensures aggregate table is fully refreshed on each run
- Consider partitioning by Date if query patterns favor date-based filtering
- Parquet format provides efficient columnar storage for analytical queries
- Ensure sufficient Glue DPUs for aggregation operations on large datasets
-

3. Runtime Strategy Notes

Authoring Language

- Primary Language: Python 3.10 (PySpark)
- Spark Version: 3.3.x (AWS Glue 4.0)
- Hudi Version: 0.12.x (compatible with Glue 4.0)
- Additional Libraries:
- boto3 (AWS SDK for Python) - for S3 and Glue API interactions
- awsglue library - for Glue-specific operations

Execution Strategy

- Job Type: AWS Glue ETL Job (Spark)
- Glue Version: 4.0
- Worker Type: G.1X or G.2X (recommend G.2X for Hudi operations)
- Number of Workers: 10 (adjustable based on data volume)
- Job Timeout: 2880 minutes (48 hours)
- Max Retries: 0 (manual retry preferred for data integrity)
- Job Bookmark: Enabled (for incremental processing in TR-SCD2-002)
- Execution Mode: Sequential execution of technical requirements in order:
 1. TR-INGEST-001, TR-INGEST-002 (parallel)
 2. TR-CLEAN-001 (depends on ingestion)
 3. TR-CLEAN-002 (depends on TR-CLEAN-001)
 4. TR-CATALOG-001, TR-CATALOG-002 (parallel, depend on TR-CLEAN-002)
 5. TR-SCD2-002 (depends on TR-CATALOG-001, TR-CATALOG-002)
 6. TR-SCD2-001 (triggered by TR-SCD2-002)
 7. TR-AGG-001 (depends on TR-SCD2-001)

Glue Job Type Expectations

- Job Type: Glue ETL (not Glue Streaming)
- Script Location: s3:///scripts/customer_order_pipeline.py
- Temporary Directory: s3:///temp/
- Spark UI Logs: s3:///sparkui-logs/
- CloudWatch Logs: Enabled (continuous logging)
- Metrics: Enabled (CloudWatch metrics for job monitoring)
- Security Configuration: (Optional) Reference to Glue security configuration for encryption
- Connections: None required (direct S3 access)
- Dependent JARs: Hudi JARs (automatically included in Glue 4.0)
-

4. Data Design Details

Source Paths / Source Systems

Source ID	Source Type	Source Path	Format	Schema Reference
-----	-----	-----	-----	-----
SOURCE-CUSTOMER	S3 CSV	s3://adif-sdlc/sdlc_wizard/customerdata/	CSV	TR-INGEST-001 Input Schema
SOURCE-ORDER	S3 CSV	s3://adif-sdlc/sdlc_wizard/orderdata/	CSV	TR-INGEST-002 Input Schema
SOURCE-ORDERSUMMARY-EXISTING	Hudi Table	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Hudi	TR-SCD2-001 Output Schema

- Note: All source paths are exactly as specified in the FRD. No intermediate sources, staging tables, or alternative sources are introduced.

Target Paths / Target Tables

Target Type	Target Path	Format	Catalog Database	Catalog Table
-----	-----	-----	-----	-----
3 Parquet + Catalog	s3://adif-sdlc/curated/sdlc_wizard/customer/	Parquet	gen_ai_poc_databricksco	sdlc_wizard_customer
3 Parquet + Catalog	s3://adif-sdlc/curated/sdlc_wizard/order/	Parquet	gen_ai_poc_databricksco	sdlc_wizard_order
3 Hudi + Catalog	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Hudi (COW)	gen_ai_poc_databricksco	ordersummary
3 Parquet + Catalog	s3://adif-sdlc/analytics/customeraggregatespend/	Parquet	gen_ai_poc_databricksco	customeraggregatespend

Catalog Registration Expectations

Table Name	Database	Registration Method	Update Frequency	Queryable via Athena
-----	-----	-----	-----	-----
sdlc_wizard_customer	gen_ai_poc_databricksco	Glue DynamicFrame write or Glue API	On each job run (overwrite)	Yes
sdlc_wizard_order	gen_ai_poc_databricksco	Glue DynamicFrame write or Glue API	On each job run (overwrite)	Yes
ordersummary	gen_ai_poc_databricksco	Hudi Hive Sync (automatic)	On each job run (upsert)	Yes
customeraggregatespend	gen_ai_poc_databricksco	Glue DynamicFrame write or Glue API	On each job run (overwrite)	Yes

Partitioning Expectations

	Table Name	Partitioning Strategy	Rationale	
	-----	-----	-----	
	sdlc_wizard_customer	None (non-partitioned)	Small dimension table; partitioning not required	
	sdlc_wizard_order	None (non-partitioned)	FRD does not specify partitioning; can be added later if needed	
	ordersummary	None (non-partitioned)	Hudi manages internal partitioning; external partitioning not specified in FRD	
	customeraggregatespend	None (non-partitioned)	FRD does not specify partitioning; can be added by Date if query patterns require	

- Note: Partitioning can be added in future iterations based on query performance analysis. Current design follows FRD specifications which do not mandate partitioning.

File Formats / Table Formats

Table Name	Storage Format	Compression	Rationale
-----	-----	-----	-----
sdlc_wizard_customer	Parquet	Snappy	Columnar format optimized for Athena queries; efficient compression
sdlc_wizard_order	Parquet	Snappy	Columnar format optimized for Athena queries; efficient compression
ordersummary	Hudi (COPY_ON_WRITE)	Snappy	SCD Type 2 requires versioning and time-travel; Hudi provides ACID transactions
customeraggregatespend	Parquet	Snappy	Analytical table optimized for read-heavy queries; columnar format

Update / Overwrite / Append Behavior

Table Name	Write Mode	Behavior	Idempotency
-----	-----	-----	-----
sdlc_wizard_customer	Overwrite	Full table replacement on each run	Yes (same input produces same output)
sdlc_wizard_order	Overwrite	Full table replacement on each run	Yes (same input produces same output)
ordersummary	Upsert (Hudi)	Insert new records; update existing records based on key	Yes (Hudi handles deduplication)
customeraggregatespend	Overwrite	Full table replacement on each run	Yes (recalculated from ordersummary)

- Idempotency Notes:
- All operations are designed to be idempotent (rerunnable without side effects)
- Overwrite mode ensures clean state on each run
- Hudi upsert mode ensures no duplicate active records
- Job can be safely retried in case of failure
-

5. Processing Details

Data Quality Rules

Rule ID	Rule Description	Applied To	Implementation	
-----	-----	-----	-----	
DQ-001	Remove records with string literal "Null" (case-sensitive)	Customer, Order	Filter: col != "Null" for all columns	
DQ-002	Remove records with actual NULL values	Customer, Order	Filter: col.isNull() for all columns	
DQ-003	Remove duplicate records (all columns match)	Customer, Order	DataFrame.dropDuplicates() on all columns	
DQ-004	Validate PricePerUnit is parseable as Decimal(10,2)	Order	Schema enforcement during CSV read	
DQ-005	Validate Qty is parseable as Integer	Order	Schema enforcement during CSV read	
DQ-006	Ensure at least one record remains after cleaning	Customer, Order	Count validation; fail if count = 0	

Deduplication Rules

Entity	Deduplication Key	Method	Retention Logic	
-----	-----	-----	-----	
Customer	All columns (CustId, Name, EmailId, Region)	dropDuplicates()	Retain first occurrence (Spark default)	
Order	All columns (OrderId, ItemName, PricePerUnit, Qty, Date, CustId)	dropDuplicates()	Retain first occurrence (Spark default)	

- Note: Deduplication is performed after null removal to ensure clean comparison.

Join Rules

Join ID	Left Table	Right Table	Join Type	Join Key	Purpose	
-----	-----	-----	-----	-----	-----	
JOIN-001	df_customer_clean	df_order_clean	Inner	CustId	Create order summary with customer details	
JOIN-002	df_order_clean	df_changed_customers	Inner	CustId	Filter orders for changed customers (incremental)	

- Join Validation:
- JOIN-001: If join produces zero records, log warning and skip SCD2 processing
- JOIN-002: Used only in incremental mode (TR-SCD2-002)

Aggregation Rules

Aggregation ID	Source	Group By	Aggregation Function	Output Column	
-----	-----	-----	-----	-----	
AGG-001	ordersummary (active records)	Name, Date	SUM(PricePerUnit × Qty)	TotalAmount	

- Aggregation Details:
- Pre-aggregation Filter: IsActive = True (only current records)
- Calculation: TotalAmount = PricePerUnit × Qty (per order line)
- Aggregation Level: Customer Name + Date
- Output: One row per customer per date with total spending

SCD / History Tracking Rules

	Implement

ed with IsActive=True, StartDate=current_timestamp, EndDate=null	Direct i
close existing (IsActive=False, EndDate=current_timestamp) and insert new (IsActive=True, StartDate=current_timestamp, EndDate=null)	Two-st
: no action	Exclud
ness: CustId + OrderId must be unique for active records	Hudi en
OpTs determines which record to keep on conflict	Hudi us
EndDate <= EndDate for all closed records	Post-w

- SCD Type 2 Column Definitions:
- IsActive (Boolean): True = current/active record, False = historical/closed record
- StartDate (Timestamp): Effective start timestamp for the record version
- EndDate (Timestamp): Effective end timestamp for the record version (null for active records)
- OpTs (Timestamp): Operation timestamp used by Hudi for precombine logic

Null Handling

Null Type	Handling Strategy	Applied To	
-----	-----	-----	
String literal "Null" (case-sensitive)	Remove entire record	Customer, Order (all columns)	
Actual NULL value	Remove entire record	Customer, Order (all columns)	
NULL in SCD2 EndDate	Expected for active records (IsActive=True)	ordersummary	
NULL in other columns	Not allowed after cleaning	All tables post-cleaning	

Type Casting Expectations

Column	Source Type	Target Type	Casting Method	Validation	
-----	-----	-----	-----	-----	
CustId	String (CSV)	String	Direct	None	
Name	String (CSV)	String	Direct	None	
EmailId	String (CSV)	String	Direct	None	
Region	String (CSV)	String	Direct	None	
OrderId	String (CSV)	String	Direct	None	
ItemName	String (CSV)	String	Direct	None	
PricePerUnit	String (CSV)	Decimal(10,2)	Schema enforcement during read	Must be parseable; fail on error	
Qty	String (CSV)	Integer	Schema enforcement during read	Must be parseable; fail on error	
Date	String (CSV)	String	Direct (no date parsing)	Stored as string; parsing deferred to query time	
IsActive	N/A	Boolean	Literal value (True/False)	N/A	
StartDate	N/A	Timestamp	current_timestamp()	N/A	
EndDate	N/A	Timestamp	current_timestamp() or null	N/A	

OpTs	N/A	Timestamp	current_timestamp()	N/A	
TotalAmount	Calculated	Decimal(20,2)	PricePerUnit × Qty	Result of multiplication	

- Type Casting Notes:
- Date column is kept as String to avoid parsing errors; date parsing can be handled at query time in Athena
- Decimal precision for TotalAmount (20,2) accommodates large aggregated values
- Timestamp columns use UTC timezone (Spark default)
-

6. Traceability Matrix

FRD to TRD Requirement Mapping

	FRD Requirement ID	TRD Requirement ID(s)	Mapping Notes	
	-----	-----	-----	
	FR-INGEST-001	TR-INGEST-001	Direct 1:1 mapping for customer data ingestion	
	FR-INGEST-002	TR-INGEST-002	Direct 1:1 mapping for order data ingestion	
	FR-CLEAN-001	TR-CLEAN-001	Direct 1:1 mapping for null removal	
	FR-CLEAN-002	TR-CLEAN-002	Direct 1:1 mapping for duplicate removal	
	FR-CATALOG-001	TR-CATALOG-001	Direct 1:1 mapping for customer table catalog registration	
	FR-CATALOG-002	TR-CATALOG-002	Direct 1:1 mapping for order table catalog registration	
	FR-SCD2-001	TR-SCD2-001	Direct 1:1 mapping for SCD Type 2 implementation	
	FR-SCD2-002	TR-SCD2-002	Direct 1:1 mapping for incremental processing logic	
	FR-AGG-001	TR-AGG-001	Direct 1:1 mapping for customer aggregate spend table	

Source Fidelity Mapping (FRD Source → TRD Source)

FRD Source	TRD Source	Status	
-----	-----	-----	
s3://adif-sdlc/sdlc_wizard/customerdata/	s3://adif-sdlc/sdlc_wizard/customerdata/	✓ Identical	
s3://adif-sdlc/sdlc_wizard/orderdata/	s3://adif-sdlc/sdlc_wizard/orderdata/	✓ Identical	
s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (existing)	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	✓ Identical	

- Source Fidelity Verification:
- All sources in TRD exactly match sources specified in FRD
- No intermediate sources, staging tables, or alternative sources introduced
- Source paths preserved from FRD → TRD → Code generation

Technical Requirement Dependencies

	TRD Requirement	Depends On	Dependency Type	
	-----	-----	-----	
	TR-CLEAN-001	TR-INGEST-001, TR-INGEST-002	Sequential (data dependency)	
	TR-CLEAN-002	TR-CLEAN-001	Sequential (data dependency)	
	TR-CATALOG-001	TR-CLEAN-002	Sequential (data dependency)	
	TR-CATALOG-002	TR-CLEAN-002	Sequential (data dependency)	
	TR-SCD2-002	TR-CATALOG-001, TR-CATALOG-002	Sequential (data dependency)	
	TR-SCD2-001	TR-SCD2-002	Sequential (triggered by incremental logic)	
	TR-AGG-001	TR-SCD2-001	Sequential (data dependency)	

•

7. ETL Mapping Requirements

Source: s3://adif-sdlc/sdlc_wizard/customerdata/ (CSV)

• Source Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

Source: s3://adif-sdlc/sdlc_wizard/orderdata/ (CSV)

• Source Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	String	Variable	No	
	CustId	String	Variable	No	

Target: gen_ai_poc_databrickscoe.sdlc_wizard_customer (Parquet)

• Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	-----	-----	-----	-----	-----	
	CustId	String	Variable	No	Direct map from customerdata.CustId	

	Name	String	Variable	No	Direct map from customerdata.Name	
	EmailId	String	Variable	No	Direct map from customerdata.EmailId	
	Region	String	Variable	No	Direct map from customerdata.Region	

- Transformation: Data quality filters applied (null removal, deduplication); no column transformations.

Target: gen_ai_poc_databrickscoe.sdlc_wizard_order (Parquet)

- Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	-----	-----	-----	-----	-----	
	OrderId	String	Variable	No	Direct map from orderdata.OrderId	
	ItemName	String	Variable	No	Direct map from orderdata.ItemName	
	PricePerUnit	Decimal	10,2	No	Direct map from orderdata.PricePerUnit	
	Qty	Integer	-	No	Direct map from orderdata.Qty	
	Date	String	Variable	No	Direct map from orderdata.Date	
	CustId	String	Variable	No	Direct map from orderdata.CustId	

- Transformation: Data quality filters applied (null removal, deduplication); no column transformations.

Target: gen_ai_poc_databrickscoe.ordersummary (Hudi)

- Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	-----	-----	-----	-----	-----	
	CustId	String	Variable	No	Direct map from customerdata.CustId (via join)	
	OrderId	String	Variable	No	Direct map from orderdata.OrderId (via join)	
	Name	String	Variable	No	Direct map from customerdata.Name (via join)	
	EmailId	String	Variable	No	Direct map from customerdata.EmailId (via join)	
	Region	String	Variable	No	Direct map from customerdata.Region (via join)	
	ItemName	String	Variable	No	Direct map from orderdata.ItemName (via join)	
	PricePerUnit	Decimal	10,2	No	Direct map from orderdata.PricePerUnit (via join)	
	Qty	Integer	-	No	Direct map from orderdata.Qty (via join)	
	Date	String	Variable	No	Direct map from orderdata.Date (via join)	
	IsActive	Boolean	-	No	Derived: True for new/updated records, False for closed records	
	StartDate	Timestamp	-	No	Derived: current_timestamp() for new/updated records	
	EndDate	Timestamp	-	Yes	Derived: null for active records, current_timestamp() for closed records	
	OpTs	Timestamp	-	No	Derived: current_timestamp() for all operations	

- Transformation:

- Join customerdata and orderdata on CustId
- Add SCD Type 2 columns (IsActive, StartDate, EndDate, OpTs)
- Implement SCD Type 2 logic (close existing, insert new)

Target: gen_ai_poc_databrickscoe.customeraggregatespend (Parquet)

- Target Schema:

Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping
-----	-----	-----	-----	-----
e	String	Variable	No	Direct map from ordersummary.Name (active records only)
Amount	Decimal	20,2	No	Derived: SUM(ordersummary.PricePerUnit × ordersummary.Qty) grouped by Name,
	String	Variable	No	Direct map from ordersummary.Date (active records only)

- Transformation:
- Filter ordersummary for IsActive = True
- Calculate TotalAmount = PricePerUnit × Qty per order
- Aggregate SUM(TotalAmount) grouped by Name, Date
-

8. Assumptions

The following assumptions are made due to missing information in the FRD:

1. Data Types: Specific data types for source columns are assumed based on typical usage patterns (e.g., CustId as String, PricePerUnit as Decimal(10,2), Qty as Integer).
2. Date Format: The Date column in order data is assumed to be in a parseable string format (e.g., YYYY-MM-DD). Actual date parsing is deferred to query time in Athena.
3. CSV Delimiter: Comma (,) is assumed as the CSV delimiter. If different, runtime parameter can be adjusted.
4. File Naming: CSV files are assumed to follow consistent naming patterns (e.g., customer_.csv, order_.csv). Specific patterns are not defined in FRD.
5. Hudi Partitioning: Non-partitioned Hudi table is assumed. Partitioning can be added later if performance requires.
6. Glue DPU Allocation: 10 DPUs recommended for Hudi operations. Actual requirement depends on data volume.
7. Timestamp Timezone: UTC timezone is assumed for all timestamp operations (Spark default).
8. First Run Behavior: On first run, all customer records are treated as "new" for SCD Type 2 processing.

9. Negative TotalAmount: Negative values in TotalAmount are logged as warnings but included in output (may indicate returns or adjustments).

10. Catalog Database Pre-existence: Database `gen\_ai\_poc\_databrickscoe` is assumed to exist before job execution.

11. IAM Permissions: Glue job is assumed to have all necessary IAM permissions (S3 read/write, Glue Catalog access, CloudWatch logging).

12. Hudi Compaction: Hudi compaction is not explicitly configured. Default Hudi behavior applies.

13. Job Retry Strategy: Manual retry is preferred over automatic retries to ensure data integrity.

14. Athena Query Patterns: Specific query patterns are not defined; table design optimizes for general analytical queries.

15. Data Volume: Processing logic is designed for moderate data volumes. For very large datasets (TB+), additional optimizations (partitioning, bucketing) may be required.

-

- Document Version: 1.0
- Last Updated: 2025-01-10
- Status: Ready for Code Generation
- Prepared By: AWS Glue Solution Engineer (AI Assistant)